

[Home](#) / [Blog](#) / Using ``cargo test`` for embedded testing with ``panic-probe``

CODING

Using ``cargo test`` for embedded testing with ``panic-probe``

Jonas

September 2020

ARTICLE

Using ``cargo test`` for embedded testing with ``panic-probe``

Published on September 01, 2020 4 min read

[Embedded](#) / [Tooling](#) / [Probe-run](#) / [Knurling-rs](#)

We're happy to announce the newest crate published as part of our [knurling](#) project: `panic-probe`.

`panic-probe` is a simple panic and fault handler that integrates into [probe-run](#), our host-side tooling for embedded programs. When using `panic-probe` in your firmware, `probe-run` will detect both Rust panics as well as HardFaults raised by the Cortex-M processor, and exit with an error code after printing a stack backtrace.

Using `panic-probe` is as easy as using any other panic handler: Simply add it to your `Cargo.toml` and `use` it inside your embedded application:

```
[dependencies.panic-probe]
git = "https://github.com/knurling-rs/probe-run"
branch = "main"
```

```
use panic_probe as _; // <- link the panic handler
```

Note that by default, `panic-probe` will *not* display the panic message, since that means linking in a lot of formatting code from `core::fmt`, which is fairly heavy.

If you do want to see the panic message, you can enable either the `print-rtt` or the `print-defmt` features. These will use `rtt-target` or `defmt` to display the panic message, respectively.

(currently, `panic-probe` is only available as a git dependency – we will publish it to crates.io along with [defmt](#) once that is ready)

1.0

Running on-device tests with `cargo test`

microcontroller. With `panic-probe`, you can actually take it a step further by using it for `cargo test` as well!

The easiest way to get started with this is to use our [crate template](#). It contains a `testsuite` crate that collects integration tests to be run on the device. You can run `cargo test -p testsuite` to run them.

Using the template, an example test (stored somewhere in `testsuite/tests/`) could look like this:

```
use template as _; // defmt transport + `panic-probe` panic
use nrf52840_hal::{
    pac::{Peripherals, TEMP},
    Temp,
};

fn test_temp(temp: TEMP) {
    defmt::info!("testing temperature sensor...");
    let mut temp = Temp::new(temp);
    let reading = temp.measure();

    assert!(reading > -10, "temp = {}°C. that's too cold!", reading);
    assert!(reading < 50, "temp = {}°C. that's too hot!", reading);
}

#[entry]
fn main() -> ! {
    let periph = Peripherals::take().unwrap();

    defmt::info!("running on-device tests...");

    test_temp(periph.TEMP);
}
```

This test will take a temperature reading using the temperature sensor integrated into the nRF52840, and assert that the reading is between -10°C and 50°C.

The output of the (failing) test would now look similar to this:

```
$ cargo test -p testsuite
  Compiling testsuite v0.1.0 (/path/to/app-template/testsuite)
  Finished test [optimized + debuginfo] target(s) in 0.31s
  Running target/thumbv7em-none-eabihf/debug/deps/testsuite-6
flashing program ..
DONE
resetting device
0.000000 INFO running on-device tests...
└─ tests/test.rs:25
0.000001 INFO testing temperature sensor...
└─ tests/test.rs:12
0.000002 ERROR panicked at 'temp = 63°C. that's too hot!',
testsuite/tests/test.rs:18:5
└─ probe-run/panic-probe/src/lib.rs:151
stack backtrace:
 0: 0x00002aee - __bkpt
 1: 0x000050e6 - panic_probe::imp::__cortex_m_rt_HardFault
 2: 0x0000539a - HardFault
    <exception entry>
 3: 0x00002b00 - __udf
 4: 0x00004f6e - cortex_m::asm::udf
 5: 0x00004fac - rust_begin_unwind
 6: 0x00002d44 - core::panicking::panic_fmt
 7: 0x00000924 - test::test_temp
 8: 0x000009e8 - test::__cortex_m_rt_main
 9: 0x0000092e - main
10: 0x00005378 - ResetTrampoline
```

...and Cargo will correctly detect the test as failing and exit accordingly. When the test passes, it should look like this instead:

```
$ cargo test -p testsuite
   Compiling testsuite v0.1.0 (/path/to/app-template/testsu
   Finished test [optimized + debuginfo] target(s) in 0.26s
   Running target/thumbv7em-none-eabihf/debug/deps/test-6
flashing program ..
DONE
resetting device
0.000000 INFO running on-device tests...
└─ tests/test.rs:24
0.000001 INFO testing temperature sensor...
└─ tests/test.rs:12
stack backtrace:
 0: 0x00002a6a - __bkpt
 1: 0x000015fe - template::exit
 2: 0x000009a2 - test::__cortex_m_rt_main
 3: 0x000008e2 - main
 4: 0x000052f4 - ResetTrampoline
 5: 0x000052ea - Reset
```

2.0

Sponsor this work



Sponsor Knurling on GitHub Sponsors

We build modern tooling, libraries, and learning material for embedded systems. We make it easy to get a grip on bare metal!

 [Sponsor](#)



Knurling

A tool set to develop embedded applications faster.

[DETAILS](#)

❤ [Sponsor](#)

Do you need help with coding?

[Get in touch](#)

[Get updates](#)

You might be also interested in these.

How we built our 2025 Embedded...

Cortex-r / Cortex-m / Arm
Jonathan
March 2025

READ MORE 

SEGGER J-Trace and Ferrocene

Segger / Stm32 / Cortex-m
Jessie
March 2025

READ MORE 



Rust on Cortex-M52

Lauterbach / Nucleo
Jessie
March 2025

READ MORE 

TRAINING

GITHUB

GET IN TOUCH

RUST EXPERTS

LINKEDIN

IMPRINT

FERROCENE

MASTODON

PRIVACY POLICY

OPEN SOURCE

TWITTER

BLOG

